Khoa học Dữ liệu

Notebook: Xử lý và trực quan hóa dữ liệu nhiều chiều
Năm học: 2025-2026

In [1]:
```
# Import thư viện
import pandas as pd
import numpy as np
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remou
nt=True).
```

File dữ liệu **oceanic_basalts.csv** chứa thông tin của nghiên cứu về thành phần khoáng chất có trong trầm tích đáy đại dương. Các mẫu được thu thập từ nhiều khu vực đại dương trên toàn thế giới. Dữ liệu bao gồm các biến

- affinity: loại trầm tích basalts, bao gồm 3 loại: island arc basalts (IAB - bazan vòng cung đảo); mid oceanic ridge basalts (MORB - bazan sống núi giữa đại dương); ocean island basalts (OIB - bazan đảo đại dương)
- SiO2: hàm lượng % SiO2 trong mẫu
- TiO2: hàm lượng % TiO2 trong mẫu
- Al2O3: hàm lượng % Al2O3 trong mẫu
- CaO: hàm lượng % CaO trong mẫu
- MgO: hàm lượng % MgO trong mẫu
- MnO: hàm lượng % MnO trong mẫu
- K2O: hàm lượng % K2O trong mẫu
- Na2O: hàm lượng % Na2O trong mẫu

In [2]:
```
url = '/content/drive/MyDrive/course_Data_Science_for_Geoscience/Week 1/Dataset/oceanic_basalts.csv'
df = pd.read_csv(url)
```

In [3]:
```
df.head(5)
```

Out[3]:

	affinity	SiO2	TiO2	Al2O3	CaO	MgO	MnO	K2O	Na2O
0	OIB	46.10	2.38	14.24	10.45	9.25	0.15	0.63	3.10
1	OIB	45.68	2.87	13.09	10.12	10.84	0.18	1.13	3.38
2	OIB	45.07	3.29	12.17	10.82	11.40	0.18	1.28	3.18
3	OIB	45.00	3.70	12.10	10.80	8.59	0.17	1.04	2.62
4	OIB	45.60	3.68	10.70	11.90	10.20	0.19	0.99	2.14

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 646 entries, 0 to 645
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   affinity  646 non-null    object
 1   SiO2      646 non-null    float64
 2   TiO2      646 non-null    float64
 3   Al2O3     646 non-null    float64
 4   CaO       646 non-null    float64
 5   MgO       646 non-null    float64
 6   MnO       646 non-null    float64
 7   K2O       646 non-null    float64
 8   Na2O      646 non-null    float64
dtypes: float64(8), object(1)
memory usage: 45.6+ KB
```

1. Biểu đồ phân tán

1.1 Biểu đồ phân tán với dữ liệu 2 chiều

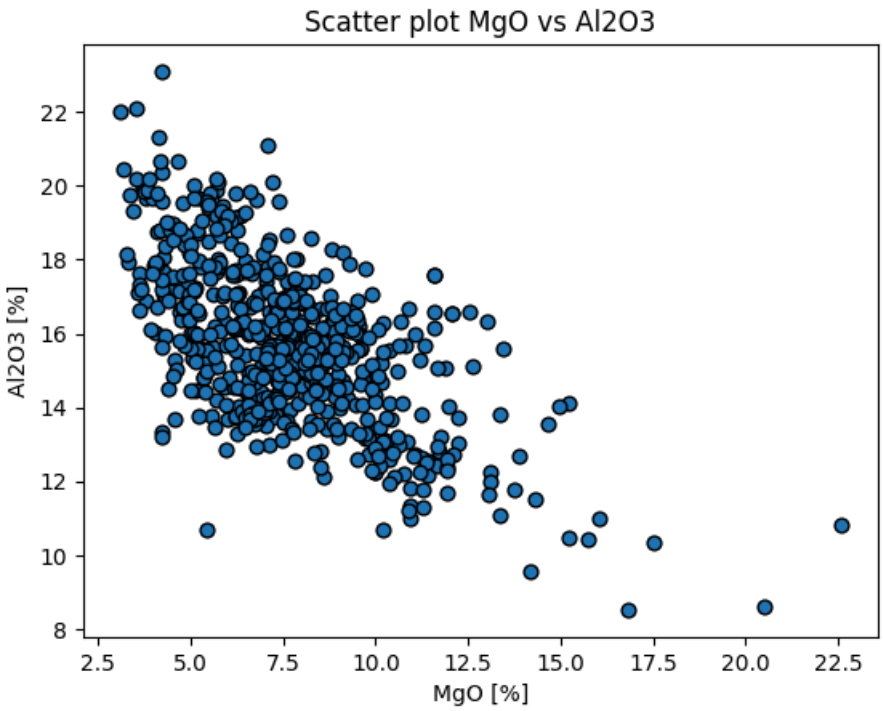
Biểu đồ phân tán là một trong những công cụ trực quan hóa cơ bản và quan trọng nhất khi phân tích mối quan hệ giữa hai biến định lượng. Mỗi điểm trên đồ thị biểu thị một quan sát (một dòng dữ liệu), với:

- Trục hoành (x-axis): biểu diễn giá trị của biến thứ nhất
- Trục tung (y-axis): biểu diễn giá trị của biến thứ hai

Biểu đồ phân tán giúp:

- Nhận biết xu hướng tuyến tính hoặc phi tuyến giữa hai biến
- Phát hiện các giá trị ngoại lai (outliers)
- Đánh giá mức độ phân tán và tương quan giữa các biến

In [5]:
```
import matplotlib.pyplot as plt

plt.scatter(
    df['MgO'],          # Trục x
    df['Al2O3'],        # Trục y
    edgecolor='black',     # Màu viền kí hiệu
    #color='#fed4e2', # Màu sắc
    s=40,               # điều chỉnh độ lớn kí hiệu
)

plt.xlabel('MgO [%]')
plt.ylabel('Al2O3 [%]')
plt.title('Scatter plot MgO vs Al2O3')
plt.show()
```



1.2 Biểu đồ phân tán kết hợp dữ liệu định tính

Với các dữ liệu kết hợp thêm các đặc điểm liên quan đến tính chất như loại trầm tích basalts. Việc thêm các đặc điểm để xác định từng tính chất giúp việc phân tích tốt hơn.

In [6]:
```
df["affinity"].value_counts()
```

Out[6]:

affinity	count
IAB	227
MORB	221
OIB	198

dtype: int64

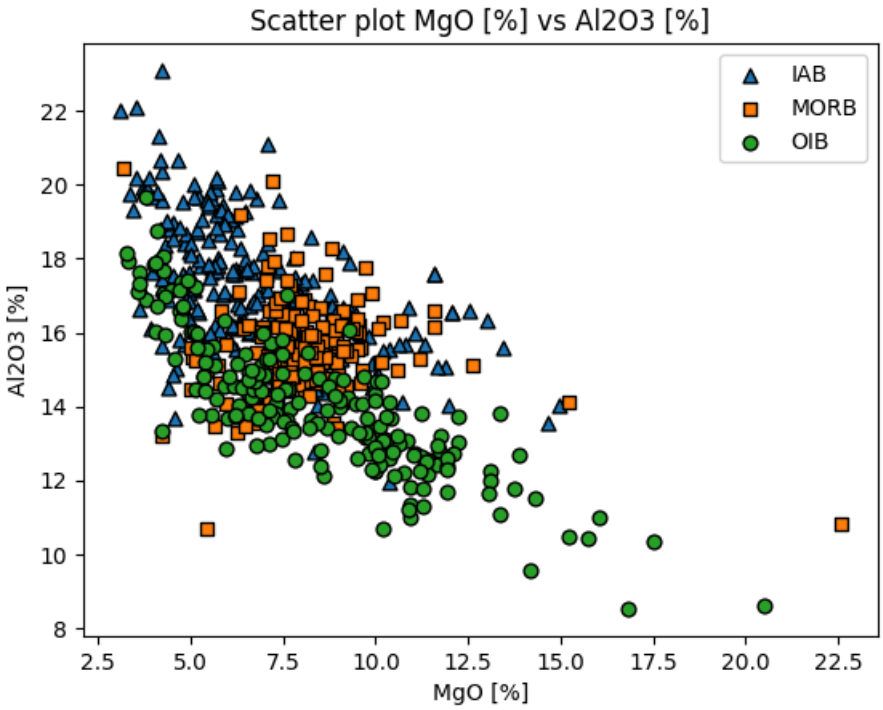
Đầu tiên cần tách dữ liệu theo từng nhóm `DataFrame` bằng `groupby`

In [7]:
```
df1 = df.groupby('affinity')
```

In [8]:
```
# Tạo dataframe tương ứng từng nhóm
IAB = df1.get_group('IAB')
MORB = df1.get_group('MORB')
OIB = df1.get_group('OIB')
```

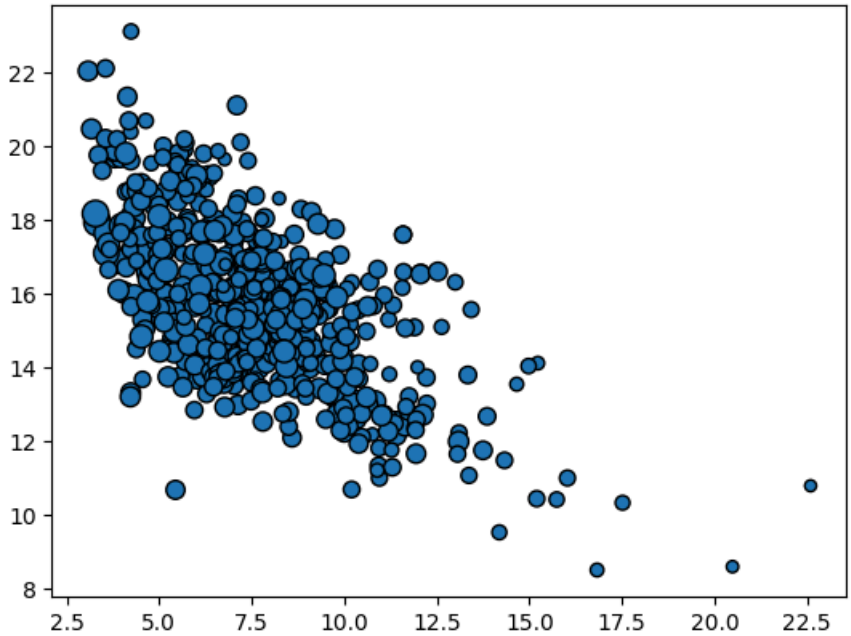
Vẽ biểu đồ phân tán với dữ liệu từng nhóm. Dùng tham số `marker =` và `s =` trong plt.scatter() để mã hóa thêm một biến định tính hoặc định lượng khác. Với các tham số như `marker` có thể tham khảo một số kí hiệu tại https://matplotlib.org/stable/api/markers_api.html. Mặc định `matplotlib` sẽ tự chọn màu sắc cho từng nhóm, tuy nhiên vẫn có thể định nghĩa màu sắc cũng như kí hiệu cho từng nhóm.

In [9]:
```
plt.scatter(
    df1.get_group('IAB')['MgO'],
    df1.get_group('IAB')['Al2O3'],
    edgecolors = 'black',
    marker = '^',
    s=40,
    label='IAB'
)
plt.scatter(
    df1.get_group('MORB')['MgO'],
    df1.get_group('MORB')['Al2O3'],
    edgecolors='black',
    marker = 's',
    s=40,
    label='MORB'
)
plt.scatter(
    df1.get_group('OIB')['MgO'],
    df1.get_group('OIB')['Al2O3'],
    edgecolors='black',
    marker = 'o',
    s=40,
    label='OIB'
)

plt.xlabel('MgO [%]')
plt.ylabel('Al2O3 [%]')
plt.title('Scatter plot MgO [%] vs Al2O3 [%]')
plt.legend()
plt.show()
```



Ngoài dữ liệu định tính, biểu đồ phân tán có thể sử dụng với 3 biến định lượng dưới độ lớn của kí hiệu nhằm thể hiện mức độ lớn nhỏ của dữ liệu biến định lượng thứ ba.

In [10]:
```
plt.scatter(
    df['MgO'], df['Al2O3'],
    edgecolors='black',
    #color='#fed4e2',
    marker = '.',
    s=df['Na2O'] * 100,
    label='D'
)
plt.show()
```

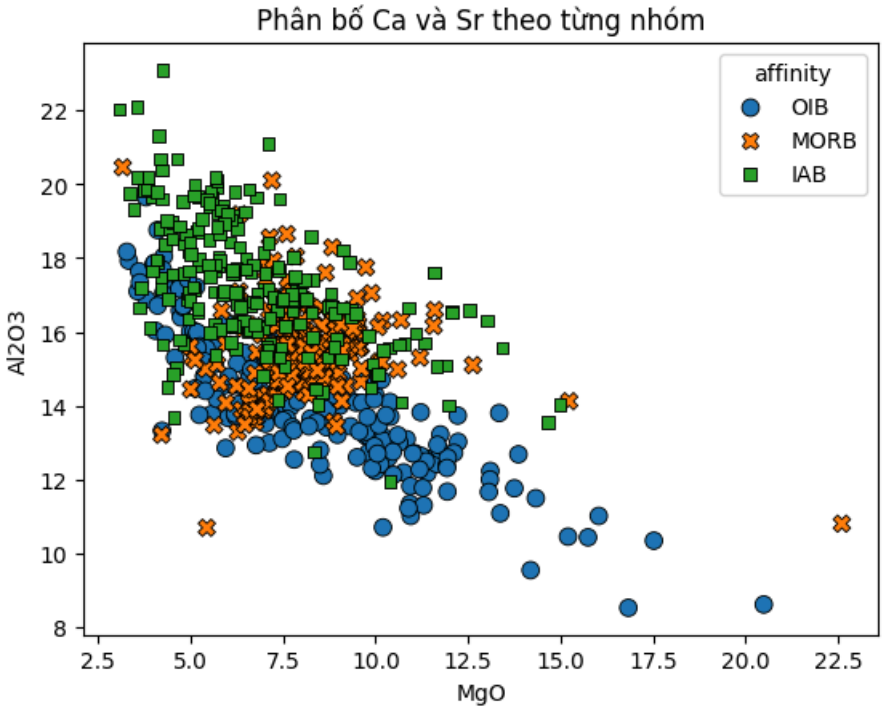


Khi trực quan hóa mối quan hệ giữa hai biến định lượng, nhưng dữ liệu lại có nhiều nhóm dữ liệu định tính (categorical groups) — ví dụ như loại đá (lithology), giới tính (gender), hay khu vực (region) — việc biểu diễn từng nhóm bằng màu sắc hoặc ký hiệu khác nhau giúp ta so sánh trực quan giữa các nhóm một cách hiệu quả hơn.

Với Matplotlib, ta có thể làm điều này thủ công bằng cách lọc và vẽ từng nhóm riêng lẻ. Tuy nhiên, cách này khá phức tạp và tốn công, đặc biệt khi dữ liệu định tính có nhiều nhóm.

Tuy nhiên, để đơn giản hóa quá trình này, ta có thể sử dụng `Seaborn (sns)` – một thư viện giúp tạo biểu đồ trực quan ngắn gọn, rõ ràng và thẩm mỹ hơn.

In [11]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    data=df,
    x='MgO', y='Al2O3',
    hue='affinity',          # phân nhóm theo SSU
    style='affinity',        # nếu muốn mỗi nhóm có ký hiệu khác nhau
    s=60,                    # độ lớn kí hiệu
    edgecolor='k'            # màu cạnh kí hiệu
)

plt.title('Phân bố Ca và Sr theo từng nhóm')
plt.legend(title='affinity')
plt.show()
```



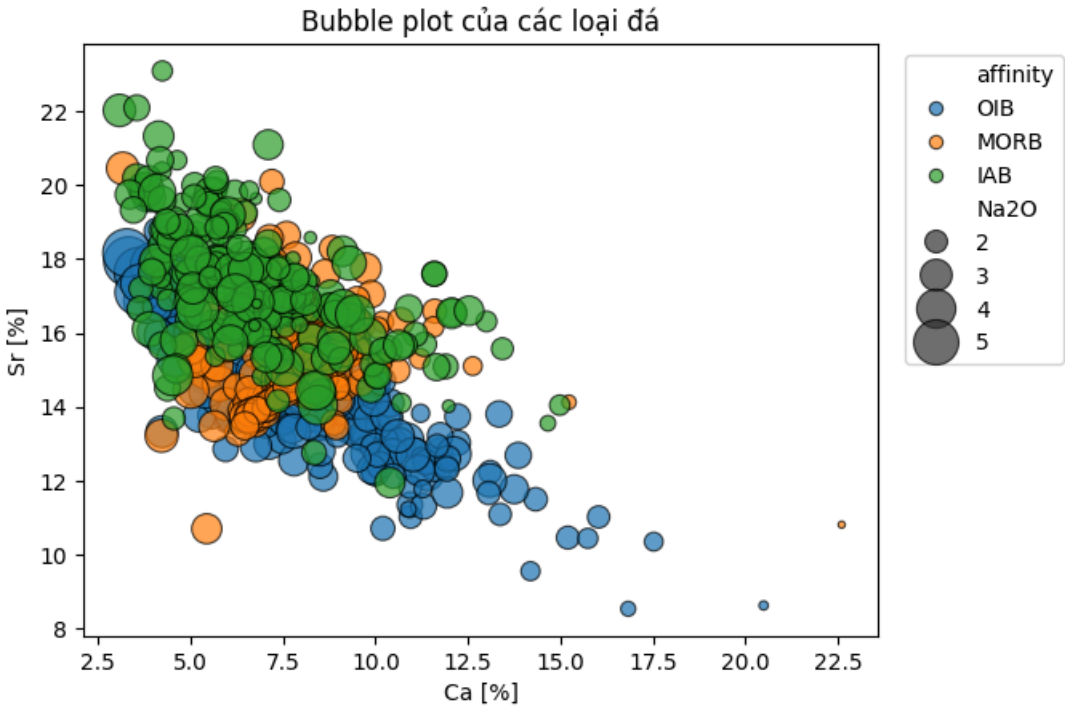
1.3 Biểu đồ bong bóng - Bubble plot

Biểu đồ bong bóng là một mở rộng của biểu đồ phân tán (scatter plot), cho phép biểu diễn ba biến định lượng cùng lúc. Trong khi hai biến đầu tiên được thể hiện qua tọa độ trên trục x và y, thì biến thứ ba được mã hóa thông qua kích thước của bong bóng (marker size). Ngoài ra, ta còn có thể dùng màu sắc (color/hue) để bổ sung thông tin về một biến định tính khác, giúp việc phân tích trở nên trực quan và sinh động hơn.

- Trục x: biểu diễn biến định lượng thứ nhất (ví dụ: Ca, pH, ...)
- Trục y: biểu diễn biến định lượng thứ hai (ví dụ: Sr, CaMg, ...)
- Kích thước bong bóng (s): phản ánh giá trị của biến thứ ba (ví dụ: Mg)
- Màu sắc (c hoặc hue): (tuỳ chọn) cho biết nhóm phân loại, ví dụ SSU.

In [12]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    data=df,
    x='MgO',
    y='Al2O3',
    size='Na2O',              # kích thước bong bóng
    hue='affinity',           # màu theo nhóm đá
    sizes=(10, 500),          # phạm vi kích thước
    alpha=0.7,
    edgecolor='k',
    #style='SSU'
```

```
)

plt.title('Bubble plot của các loại đá')
plt.xlabel('Ca [%]')
plt.ylabel('Sr [%]')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.show()
```



2. Biểu đồ mật độ hai chiều

Biểu đồ mật độ hai chiều được sử dụng để biểu diễn mật độ điểm dữ liệu trong không gian 2 chiều, giúp ta quan sát khu vực nào có tập trung dữ liệu cao (mật độ lớn) và khu vực nào thưa thớt (mật độ nhỏ).

Khác với biểu đồ phân tán (scatter plot), biểu đồ mật độ 2D không hiển thị từng điểm riêng lẻ, mà thay vào đó thể hiện phân bố xác suất chung của hai biến định lượng dưới dạng đường đồng mức (contour lines).

Tương tự với vẽ hàm mật độ với dữ liệu một chiều, ở đây vẫn sử dụng thư viện `sklearn` để ước lượng mật độ cho hai biến được chọn.

In [13]:
```
from sklearn.neighbors import KernelDensity
```

In [14]:
```
df_kde = df[['MgO', 'Al2O3']]
df_kde.head(5)
```

Out[14]:

	MgO	Al2O3
0	9.25	14.24
1	10.84	13.09
2	11.40	12.17
3	8.59	12.10
4	10.20	10.70

Tiếp theo, tính giá trị độ rộng tối ưu cho từng cột với được xác định theo công thức được đề xuất bởi Scott:

$$h_{j,opt,\mathrm{S}} = \min\left\{s_j, \mathrm{IQR}_j/1.34\right\} n^{-1/(d+4)},$$

In [15]:
```
# Tính bandwidth cho cột MgO
x1 = df_kde['MgO']
```

```
n1 = len(x1)
s1 = np.std(x1, ddof=1)
iqr1 = np.quantile(x1, 0.75) - np.quantile(x1, 0.25)

h1 = min(s1, iqr1 / 1.34) * n1 ** (-1 / (2 + 4))

print("h1 (MgO):", h1)
```

```
h1 (MgO): 0.7474952107038011
```

In [16]:
```
# Tính bandwidth cho cột Al2O3
x2 = df_kde['Al2O3']
n2 = len(x2)
s2 = np.std(x2, ddof=1)
iqr2 = np.quantile(x2, 0.75) - np.quantile(x2, 0.25)

h2 = min(s2, iqr2 / 1.34) * n2 ** (-1 / (2 + 4))

print("h2 (Al2O3):", h2)
```

```
h2 (Al2O3): 0.6662733202368346
```

Khác với vẽ hàm mật độ cho dữ liệu một chiều, với dữ liệu nhiều chiều cần đưa chuẩn hóa dữ liệu bằng việc chia các giá trị cột với độ rộng tối ưu tương ứng.

In [17]:
```
df_kde.loc[:, 'MgO'] = df_kde['MgO'] / h1
df_kde.loc[:, 'Al2O3'] = df_kde['Al2O3'] / h2
```

In [18]:
```
df_kde
```

Out[18]:

	MgO	Al2O3
0	12.374661	21.372610
1	14.501765	19.646592
2	15.250934	18.265777
3	11.491712	18.160715
4	13.645572	16.059475
...	...	...
641	9.083670	25.199868
642	10.461605	26.265497
643	9.859595	26.640719
644	4.628792	29.012118
645	7.638845	28.291693

646 rows × 2 columns

Tương tự với hàm mật độ với dữ liệu một chiều, tuy nhiên vì dữ liệu đã được chuẩn hóa với độ rộng tương ứng nên vì vậy cần khi gọi `KernelDensity` thì tham số **bandwidth** sẽ cố định là 1

In [19]:
```
X = np.array(df_kde)
```

In [20]:
```
kde = KernelDensity(kernel="gaussian", bandwidth=1.0)
kde.fit(X)
```

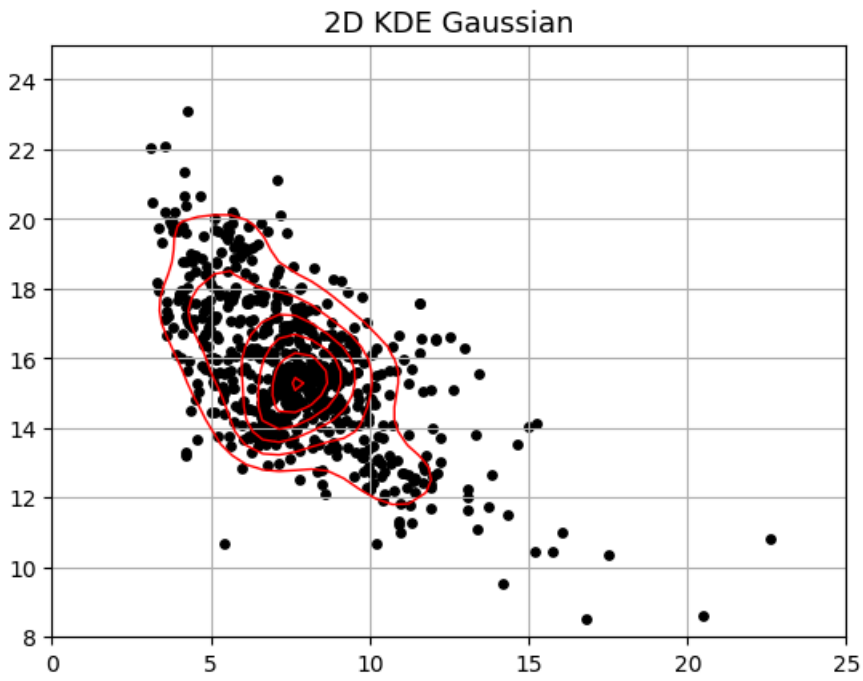
Out[20]:
```
KernelDensity
KernelDensity()
```

Tiếp theo cần tạo các điểm để có thể ước lượng Kernel Density. Khác với dữ liệu một chiều chỉ cần tạo các điểm, thì với dữ liệu nhiều chiều và đã được chuẩn hóa thì cần tạo các điểm cho trục x, trục y, sau đó chuẩn hóa để phù hợp với dữ liệu. Từ đó kết hợp lại để tạo giá trị một điểm là tọa độ trên hệ tọa độ Oxy (x, y).

In [21]:
```
# Tạo grid
x1_grid = np.linspace(0, 25, 50)
x2_grid = np.linspace(8, 25, 50)
```

```
# Chuẩn hóa các điểm cần được ước lượng
x1_scaled = x1_grid / h1
x2_scaled = x2_grid / h2

# Tạo lưới từ các điểm được chuẩn hóa
X1g, X2g = np.meshgrid(x1_scaled, x2_scaled)

# Ghép lại thành lưới 2D
grid_scaled = np.column_stack((X1g.flatten(), X2g.flatten()))
```

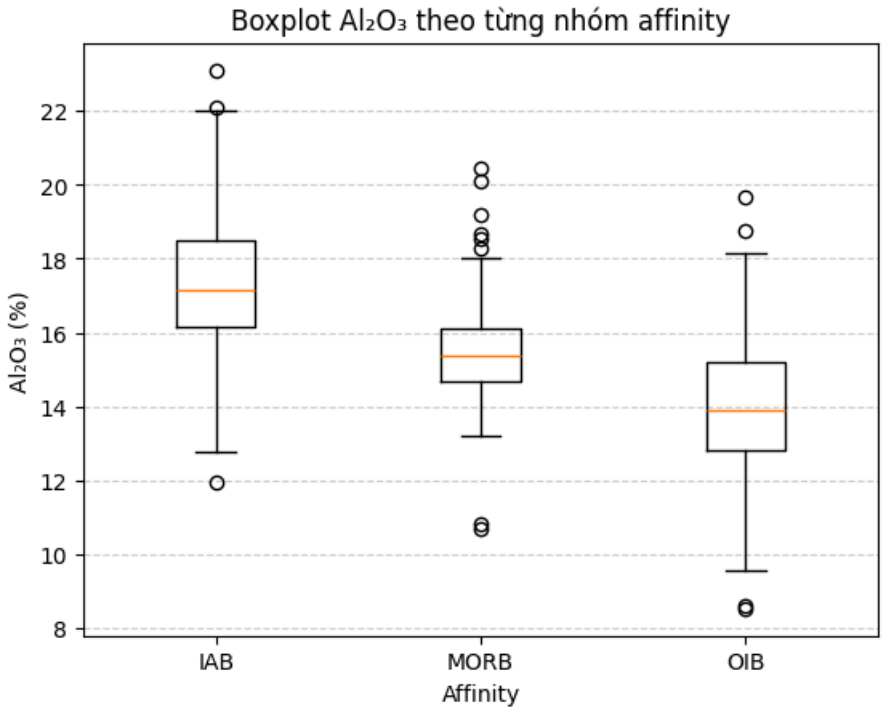
In [22]:
```
# Ước lượng mật độ
log_dens_scaled = kde.score_samples(grid_scaled)
dens_scaled = np.exp(log_dens_scaled)

# Quay lại không gian gốc
dens_true = dens_scaled.reshape(X1g.shape) / (h1 * h2)

# === VẼ CONTOUR ===
plt.contour(x1_grid, x2_grid, dens_true, colors='red', linewidths=1)
plt.scatter(df['MgO'], df['Al2O3'], s=15, c='k')
plt.title("2D KDE Gaussian", fontsize=13)
plt.grid(True)
plt.show()
```



3. Biểu đồ hộp cho nhiều nhóm

Để vẽ biểu đồ hộp cho nhiều nhóm, cần tạo một list gồm các nhóm dữ liệu. Với hàm `plt.boxplot` sẽ cho phép vẽ các thành phần trong nhóm dữ liệu này, tham số `tick_labels` sẽ là một danh sách các nhãn của nhóm, cần lưu ý là các nhãn này cần phải chính xác với thứ tự trong danh sách các nhóm dữ liệu.

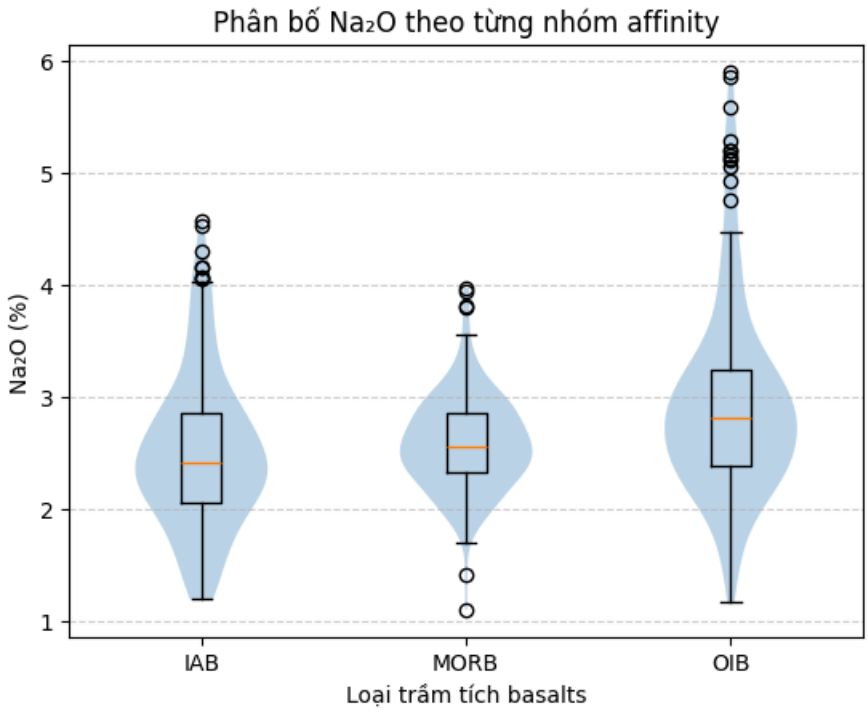
In [23]:
```
import matplotlib.pyplot as plt

# Chuẩn bị dữ liệu cho boxplot
groups = [IAB['Al2O3'], MORB['Al2O3'], OIB['Al2O3']]

# Vẽ boxplot
plt.boxplot(groups, tick_labels=['IAB', 'MORB', 'OIB'], patch_artist=False)

plt.ylabel("Al₂O₃ (%)")
plt.xlabel("Affinity")
plt.title("Boxplot Al₂O₃ theo từng nhóm affinity")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```



4. Biểu đồ violin cho nhiều nhóm

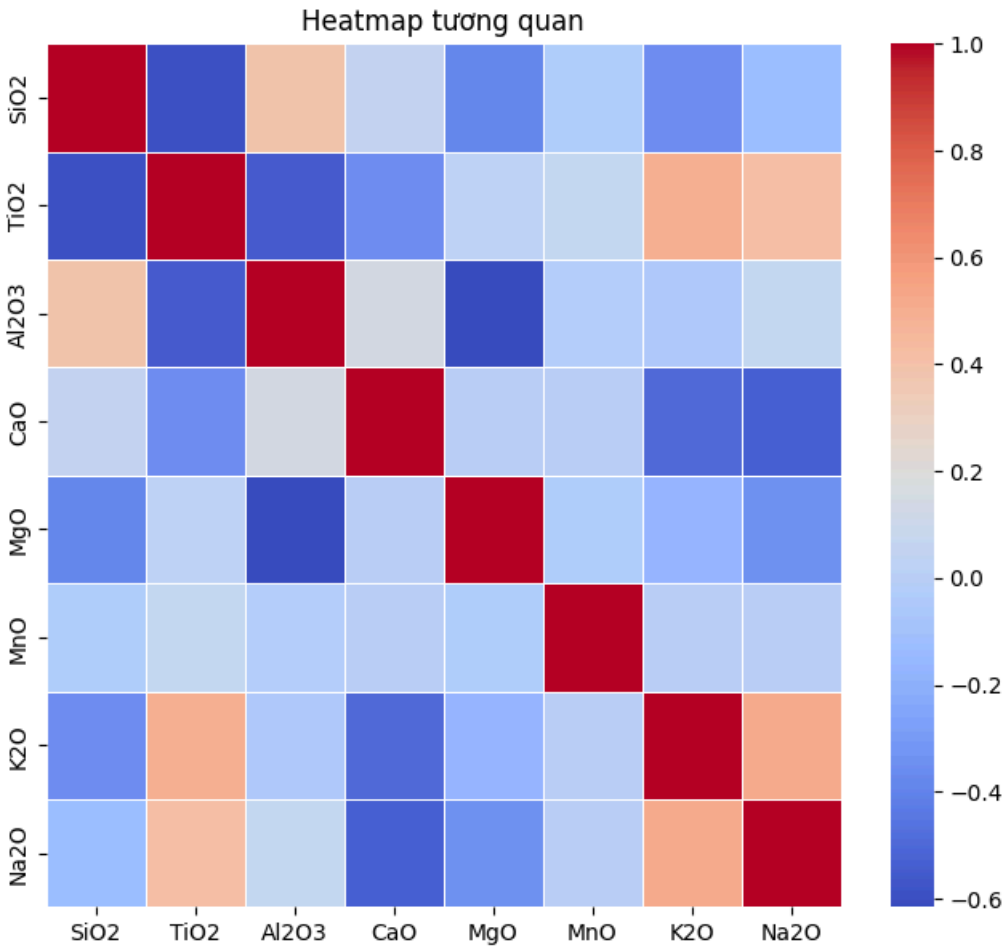
Nếu với biểu đồ hộp cho nhiều nhóm chỉ quan sát được khoảng dữ liệu cũng như quan sát được dữ liệu ngoại lai. Tuy nhiên, hình dáng phân phối của dữ liệu với từng nhóm là không thể xác định chính xác, để có thể xác định được phân phối của dữ liệu thì cần kết hợp thêm hàm mật độ xác suất Kernel Density. Từ đó, để giải quyết cả hai vấn đề, biểu đồ violin sẽ là một lựa chọn hợp lý.

In [24]:
```
import matplotlib.pyplot as plt

groups = [IAB['Na2O'], MORB['Na2O'], OIB['Na2O']]
labels = ['IAB', 'MORB', 'OIB']

# --- Violin plots ---
plt.violinplot(groups, showextrema=False)

# --- Boxplots ---
plt.boxplot(groups, widths=0.15, tick_labels=labels)

plt.xlabel("Loại trầm tích basalts")
plt.ylabel("Na₂O (%)")
plt.title("Phân bố Na₂O theo từng nhóm affinity")
plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.show()
```



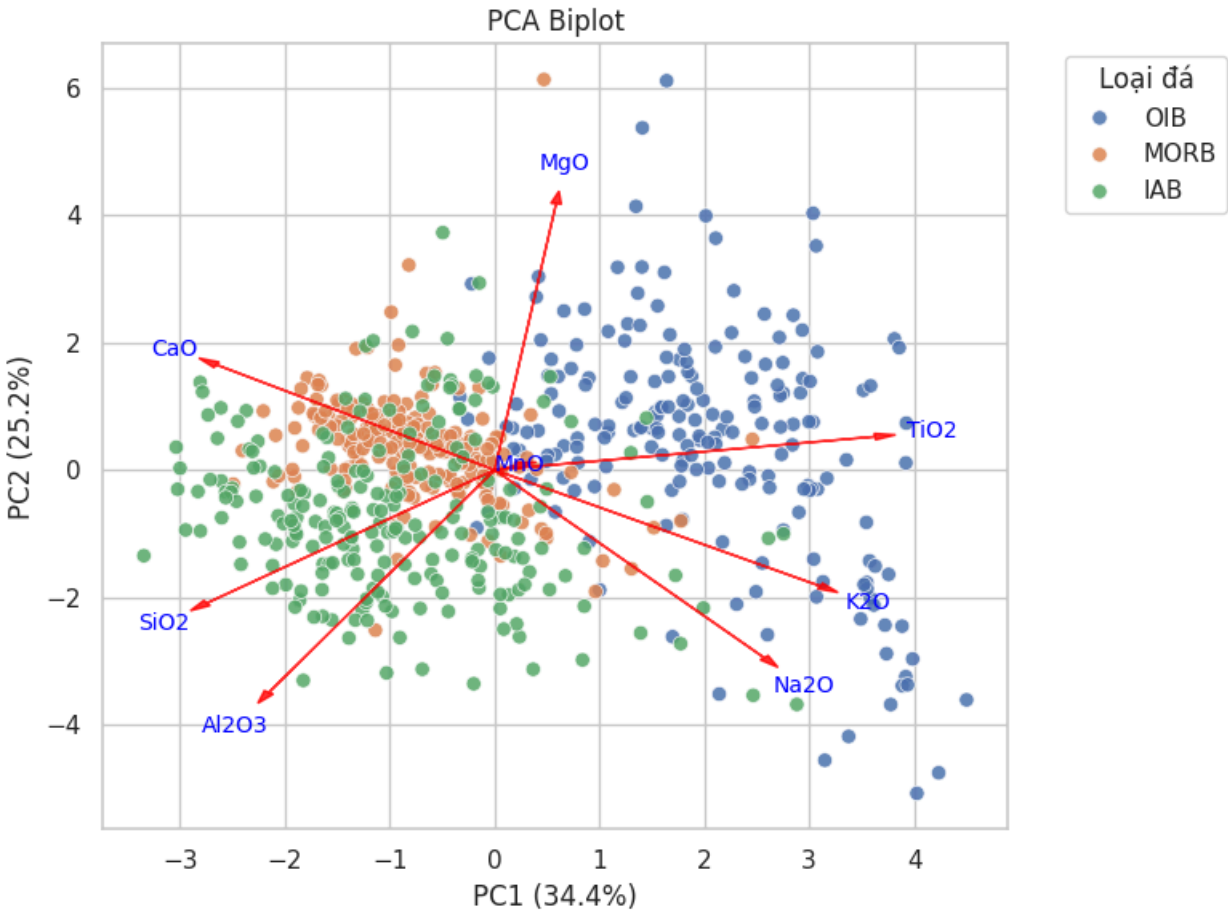
5. Biểu đồ tương quan

In [25]:
```
# --- Tính ma trận tương quan ---
corr = df.select_dtypes(include='number').corr()

# --- Vẽ heatmap ---
plt.figure(figsize=(8, 7))
sns.heatmap(corr,
            annot=False,
            cmap="coolwarm",
            fmt=".2f",
            linewidths=0.5)
plt.title("Heatmap tương quan")
plt.show()
```



6. Phương pháp giảm số chiều và biểu đồ biplot

In [26]:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

sns.set_theme(style="whitegrid")
#sns.set_palette("pastel")

# --- Lấy dữ liệu số ---
X = df.select_dtypes(include='number')

# --- Chuẩn hóa ---
X_scaled = StandardScaler().fit_transform(X)

# --- PCA ---
pca = PCA(n_components=2)
scores = pca.fit_transform(X_scaled)

# --- Gộp lại với loại đá ---
df_pca = df.copy()
df_pca['PC1'] = scores[:, 0]
df_pca['PC2'] = scores[:, 1]

# --- Hệ số phóng đại độ dài vector ---
arrow_scale = 7
loadings = pca.components_.T * arrow_scale

# --- Vẽ scatter plot (phân loại theo loại đá) ---
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=df_pca,
    x='PC1',
    y='PC2',
    hue='affinity',
    s=45,
    alpha=0.85,
    #edgecolor='black'
)
```

```python
# --- Vẽ vector tải (loadings) ---
for i, feature in enumerate(X.columns):
    plt.arrow(0, 0, loadings[i, 0], loadings[i, 1],
              color='red', alpha=0.8, head_width=0.1, head_length=0.2, linewidth=1)
    plt.text(loadings[i, 0]*1.15, loadings[i, 1]*1.15, feature,
             color='blue', fontsize=10, ha='center', va='center')


# --- Hoàn thiện ---
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.title("PCA Biplot")
plt.legend(title='Loại đá', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```